Tarea 4 · Formulación de problemas y búsqueda

Ejercicios 3.6 y 3.7 · Enunciados proporcionados de la tercera edición

Material de apoyo: presentación 7, páginas 19–20. Fecha: 3 de noviembre de 2026.

En cada formulación se distinguen el estado, el estado inicial, la prueba de meta, los sucesores y el costo. Las posiciones o los datos que el enunciado no especifica se mantienen como parámetros. Las convenciones elegidas para concretar los modelos se declaran explícitamente. [1]

3.6 a. Colorear un mapa con cuatro colores

Datos: un grafo no dirigido (R,E), donde cada vértice representa una región. Se incluye (r_i,r_j) en E si las regiones comparten un tramo de frontera; se considera que tocarse sólo en un punto no las hace adyacentes. Los colores disponibles son {1,2,3,4}. Se fija un orden de las regiones $r_1,\ldots,r_n$.

Estado: un vector $c=(c_1,\ldots,c_n)$, con $c_i\in\{0,1,2,3,4\}$. El 0 significa sin colorear. Una asignación parcial es válida si los extremos ya coloreados de cada arista tienen colores distintos.

Estado inicial: $c_0=(0,\ldots,0)$, con todas las regiones sin colorear. Prueba de meta: todos los componentes son distintos de 0 y, para toda arista $(i,j)\in E$, $c_i\neq c_j$.

Función de sucesores: elegir la primera región i con $c_i=0$. Para cada color $k\in\{1,2,3,4\}$ que no esté usado por un vecino de i ya coloreado, producir c' igual a c salvo $c'_i=k$. Si no hay color admisible, ese estado no tiene sucesores. Fijar el orden no elimina ninguna coloración completa factible: sólo evita explorar órdenes equivalentes de asignación.

Costo: 1 por colorear una región. El costo de un camino es el número de asignaciones realizadas; toda solución completa cuesta n. El procedimiento se implementa por búsqueda con retroceso. El código adjunto verifica una instancia planar K_4, que requiere cuatro colores.

3.6 b. El mono, las dos cajas y las bananas

Abstracción: el suelo se representa por un conjunto finito de zonas P y un grafo de movimientos entre ellas. Cada zona permite colocar y maniobrar con ambas cajas. Una zona no es un punto geométrico de volumen nulo. Las bananas están sobre la zona b, a 8 pies. Las cajas C1 y C2 miden 3 pies y el mono 3 pies. Se usa esa altura como alcance vertical del modelo; se abstraen la postura y los detalles de manipulación.

Estado: $s=(m,q,p_1,p_2,s_1,s_2,\text{tiene})$. m es la zona del mono; $q\in\{\text{suelo},C1,C2\}$ es su soporte; p_i es la zona de Ci; $s_i\in\{\text{suelo},\text{la otra caja}\}$ es su soporte; tiene es un booleano que indica si posee las bananas. No se permiten ciclos de soporte. Si una caja está sobre otra, comparten zona. Si el mono está sobre una caja, comparte su zona y la parte superior de esa caja debe estar libre de la otra caja.

Estado inicial elegido: $(m_0,\text{suelo},p_{10},p_{20},\text{suelo},\text{suelo},\text{Falso})$. Así se concreta la situación inicial con mono y cajas en el suelo y cajas sin apilar; las zonas m_0,p_{10},p_{20} y b se suministran al definir la instancia porque el texto no las fija. Si se especifica otra disposición inicial, se reemplaza ese estado por la configuración válida correspondiente.

Prueba de meta: tiene=Verdadero. La altura superior de una caja es 3 pies si descansa en el suelo y 6 si está sobre la otra. La base del mono está a 0 si está en el suelo o a la altura superior de su caja de soporte. Tomar las bananas requiere m=b y altura_base+3≥8: con una caja se llega a 6 pies, y con dos apiladas se puede alcanzar la altura de 8 pies.

Sucesores: se aplican únicamente los operadores cuyas precondiciones se cumplen. “Libre” significa que no hay otra caja encima; en las operaciones de suelo el mono también debe estar en el suelo.

Operador	Precondición y efecto
Caminar a z	Mono en el suelo y z adyacente a m. Cambiar m a z.
Empujar Ci a z	Mono en el suelo, $m=p_i$, Ci sobre el suelo y libre; z adyacente. Cambiar m y p_i a z.
Apilar Ci sobre Cj	Mono en el suelo; $m=p_i=p_j$; ambas cajas sobre el suelo y libres. Cambiar s_i a Cj.
Desapilar Ci	Mono en el suelo, misma zona; Ci está sobre Cj y libre. Poner Ci en el suelo de la misma zona.
Subir a Ci	Mono en el suelo, $m=p_i$ y Ci libre. Cambiar q a Ci; puede ser la caja superior de una pila.
Bajar	Mono sobre una caja. Cambiar q a suelo, conservando la zona.
Tomar bananas	tiene=Falso, m=b y altura_base+3≥8. Cambiar tiene a Verdadero.

Costo: 1 por operador. Los desplazamientos son entre zonas adyacentes y subir/bajar son operadores abstractos de escalada; el costo mide operaciones del modelo. Un plan general consiste en llevar ambas cajas a b, apilar una sobre la otra, subir a la superior y tomar las bananas. No se atribuye un número mínimo de movimientos al cuarto original, cuyas posiciones no se proporcionan.

3.6 c. Identificar un registro ilegal

Datos: registros completos $r_1,\ldots,r_n$ y un oráculo falla(C), que procesa los registros cuyos índices están en C, con el mismo formato y contexto de validación, y devuelve Verdadero si alguno es ilegal. La independencia del procesamiento permite probar subconjuntos sin cambiar la validez de cada registro. No se cortan registros por la mitad.

Estado: un conjunto ordenado no vacío C de índices candidatos, con la invariante falla(C)=Verdadero. Estado inicial: $C_0=\{1,\ldots,n\}$, ya conocido como fallido por el mensaje del programa. Prueba de meta: $|C|=1$. La invariante permite identificar su único candidato como ilegal.

Función de sucesores: si $|C|>1$, dividirlo en mitades no vacías L y R, conservando el orden original. Ejecutar falla(L). Si devuelve Verdadero, el nuevo estado es L; si devuelve Falso, es R. En este último caso, la independencia y falla(C)=Verdadero garantizan que R contiene un registro ilegal. La respuesta del oráculo determina la transición.

Costo: 1 por consulta adicional al programa; la falla inicial ya observada no se vuelve a cobrar. El tamaño de los candidatos se reduce aproximadamente a la mitad en cada consulta. Se necesitan como máximo $\lceil \log_2 n \rceil$ consultas adicionales. Esto cuenta invocaciones, no tiempo total de CPU: cada consulta puede procesar varios registros.

No hace falta suponer que existe exactamente un registro ilegal: si hay varios, el procedimiento encuentra al menos uno. Para encontrarlos todos se deben explorar también las otras mitades que fallen. La implementación se verificó con las 255 distribuciones no vacías de registros ilegales en un archivo de ocho registros; siempre identifica uno y utiliza como máximo tres consultas.

3.6 d. Jarras de 12, 8 y 3 galones

Estado: $s=(x_{12},x_8,x_3)$, con cantidades enteras que satisfacen $0\le x_{12}\le 12$, $0\le x_8\le 8$ y $0\le x_3\le 3$. Para concretar la instancia se eligen inicialmente las tres jarras vacías: $s_0=(0,0,0)$. Si se dan otros contenidos iniciales, se cambia s_0. Los trasvases se modelan hasta vaciar el origen o llenar el destino; así no se presupone la capacidad de medir directamente una cantidad parcial arbitraria.

Prueba de meta: $x_{12}=1$ o $x_8=1$ o $x_3=1$. Basta que alguna jarra contenga exactamente un galón; el contenido de las otras no invalida esa medida.

Acción	Sucesor y condición
Llenar i	Si $x_i<C_i$, sustituir x_i por C_i.
Vaciar i al suelo	Si $x_i>0$, sustituir x_i por 0.
Verter i en j	Para $i\neq j$, $t=\min(x_i, C_j-x_j)>0$. Sustituir x_i por x_i-t y x_j por x_j+t.

Costo: 1 por llenar, vaciar o trasvasar. El costo de un camino es su número de operaciones. La búsqueda en anchura encuentra una solución mínima porque todos los pasos tienen el mismo costo.

Paso	Acción	Estado (12,8,3)
0	Inicial	(0,0,0)
1	Llenar la jarra de 12	(12,0,0)
2	Verter de 12 a 8 hasta llenar la de 8	(4,8,0)
3	Verter de 12 a 3 hasta llenar la de 3	(1,8,3)

Queda un galón en la jarra de 12. Tres operaciones son mínimas desde el estado vacío: tras una operación sólo se puede disponer de una capacidad completa; en una segunda, un trasvase puede dejar diferencias $12-8=4$, $12-3=9$ u $8-3=5$, o conservar una capacidad completa al pasar de una jarra menor a otra mayor. Ninguna de esas cantidades es 1. La solución de tres pasos fue comprobada por BFS.

3.7. Camino mínimo entre obstáculos poligonales convexos

Se adopta la idealización de un robot puntual. El movimiento no puede entrar en el interior de los obstáculos, pero sí puede tocar sus vértices o recorrer sus bordes. Esta convención permite incluir los vecinos del mismo polígono y que exista una ruta mínima que se apoye en las esquinas. Los obstáculos de la escena se consideran disjuntos. [2–3]

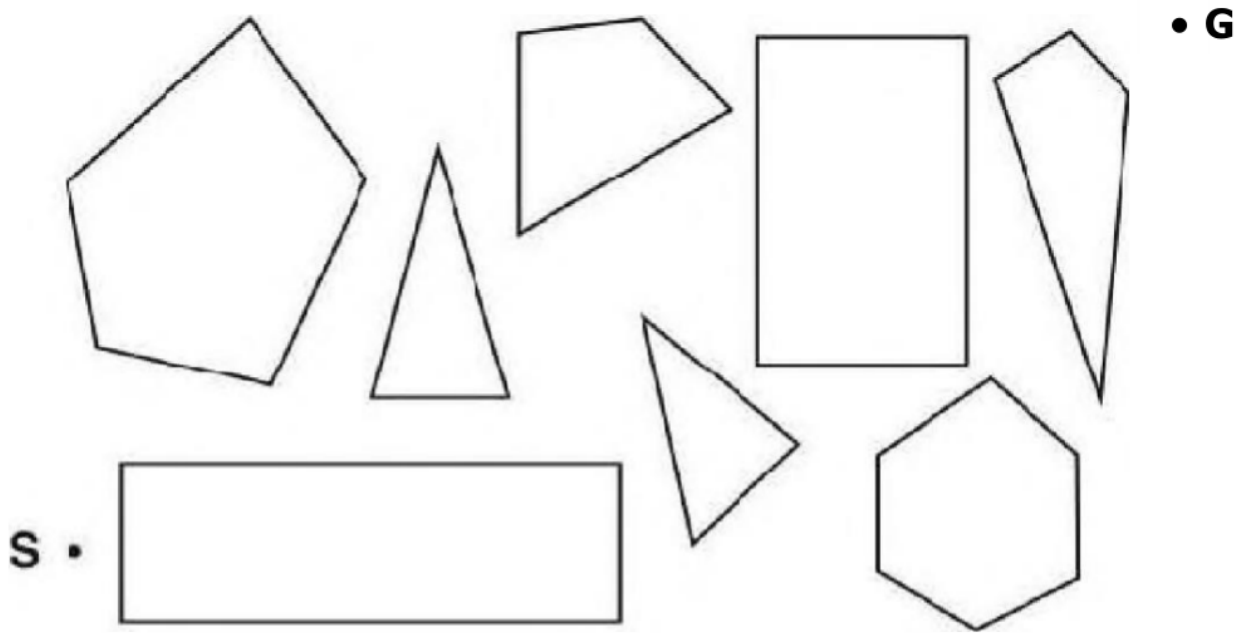


Figura 3.31 del material proporcionado: presentación 7, página 20. Se conserva la geometría del dibujo; no se le atribuyen coordenadas ni distancias numéricas que el enunciado no indica.

3.7 a. Cantidad de estados y caminos

En la formulación continua, el estado es la posición (x,y) del robot fuera de los interiores prohibidos. Hay infinitos estados no numerables: el espacio contiene zonas completas del plano. También hay infinitos caminos factibles no numerables hacia G, porque se pueden variar continuamente sus puntos de paso y sus rodeos. No basta enumerar todas las posiciones o trayectorias posibles.

3.7 b. Por qué bastan los vértices y cuántos estados quedan

En una zona libre, un tramo curvo o un quiebre se puede acortar reemplazándolo por el segmento que une sus extremos. Un giro en el interior de una arista también se puede eliminar localmente, del lado libre del obstáculo. Por tanto, los cambios de dirección de una ruta mínima sólo necesitan ocurrir en vértices de obstáculos; entre ellos, y desde S o hacia G, los tramos son rectos. Recorrer parte de un borde es un caso permitido. [2]

Se define V={S,G}∪{todos los vértices de los obstáculos}. Se conectan dos elementos de V si el segmento entre ellos no entra en ningún interior prohibido. Este es el grafo de visibilidad. Conserva al menos una ruta de longitud mínima del problema continuo bajo las convenciones anteriores. [2–3]

Obstáculo de la figura	Vértices
Pentágono superior izquierdo	5
Triángulo estrecho del centro-izquierda	3
Cuadrilátero superior, a la izquierda del rectángulo central	4
Rectángulo central	4
Cuadrilátero estrecho superior derecho	4
Triángulo inferior central	3
Hexágono inferior derecho	6
Rectángulo inferior izquierdo	4
Total de vértices de obstáculos	33
Total de estados con S y G	35

En general, si hay N vértices distintos y S y G no coinciden con ellos, el espacio tiene N+2 estados. En esta figura son 35. Hay como máximo 35×34/2=595 aristas no dirigidas; sólo se mantienen las visibles. El grafo es finito, aunque sus recorridos con repeticiones pueden ser infinitos. Con costos positivos, una ruta mínima puede elegirse sin ciclos.

3.7 c. Funciones de búsqueda y heurística

Componente	Definición implementable
Entrada	Coordenadas de S, G y vértices de cada polígono, ordenados por el contorno.
Estado inicial	El vértice S. Se almacenan índices de vértices y sus coordenadas.
GOAL_TEST(v)	Verdadero si el identificador de v es el de G.
VISIBLE(v,w)	Verdadero si el segmento cerrado [v,w] no intersecta el interior de ningún obstáculo.
ACTIONS(v)	$\{p(w)-p(v) : w \in V,\ w \neq v$ y VISIBLE(v,w)$\}$. Devuelve vectores de desplazamiento.
RESULT(v,a)	El vértice w cuya posición es p(v)+a; se conserva el destino asociado al vector.
STEP_COST(v,a,w)	$\|a\|_2 = \sqrt{(a_x^2 + a_y^2)}$, la longitud del segmento.
Costo del camino	Suma de las longitudes de los segmentos recorridos.
h(v)	$\|p(G)-p(v)\|_2$, distancia euclidiana directa hasta G.
Criterio de A*	Extraer el estado de menor f(v)=g(v)+h(v); terminar al extraer G.

ACTIONS incluye los vértices consecutivos de un mismo polígono: su arista se puede recorrer sobre el borde. Una diagonal que entra en el interior no es una acción válida. Un contacto tangencial con una esquina sí puede ser válido, pero cruzar por una esquina y continuar dentro sigue bloqueado. La prueba geométrica debe distinguir contacto de entrada al interior.

```
ACTIONS(v):
    devolver { p(w)-p(v) para w en V
               si w != v y VISIBLE(v,w) }

RESULT(v,a):
    devolver el destino asociado a (v,a)

STEP_COST(v,a,w): devolver norma_euclidiana(a)
h(v): devolver distancia_euclidiana(p(v),p(G))
```

La heurística es admisible: ignorar obstáculos no puede aumentar la distancia mínima. También es consistente por la desigualdad triangular: $h(v) \leq d(v,w)+h(w)$ para cada arista visible y $h(G)=0$. Por ello A* con estos costos y esta heurística puede encontrar una ruta de longitud mínima en el grafo. Se minimiza distancia, no número de segmentos.

Para implementar VISIBLE con polígonos convexos, el código adjunto escribe cada punto del segmento como $u+t(w-u)$, $0 \leq t \leq 1$. Para cada polígono orientado en sentido antihorario, su interior satisface que el producto cruzado con cada arista sea estrictamente positivo. Si existe un intervalo de t que satisface todas esas desigualdades, el segmento entra en el interior y se descarta. La igualdad corresponde al borde y no bloquea por sí sola.

Las coordenadas de la figura no vienen dadas como datos numéricos, por lo que aquí se define el algoritmo sin afirmar una longitud exacta para ese dibujo. La implementación utiliza coordenadas racionales e identificadores para evitar comparar vértices mediante sumas aproximadas de punto flotante. Las longitudes se evalúan numéricamente.

Comprobaciones y archivos de apoyo

soluciones.py contiene el coloreador por retroceso, los operadores del mono, la búsqueda del registro ilegal, las jarras por BFS y el grafo de visibilidad con A*. Se ejecutó y verificó una coloración de K_4, un plan válido del modelo del mono, las 255 distribuciones no vacías de registros ilegales en ocho registros, la solución mínima de tres pasos de las jarras y casos geométricos de borde, diagonal y tangencia.

La prueba de A* usa un rectángulo sintético con vértices (0,0),(2,0),(2,2),(0,2), inicio $(-1,1)$ y meta (3,1). Su resultado coincide con la longitud analítica $2+2\sqrt{2}$. Ese ejemplo sirve para verificar el código; no es una digitalización de la figura 3.31. El archivo resultados/verificacion.json conserva los resultados. El programa usa sólo la biblioteca estándar de Python y se ejecuta con python soluciones.py.

Referencias

[1] Material proporcionado: 7-Solución de Problemas.pdf, páginas 19–20. Enunciados 3.6 y 3.7 y figura 3.31.

AIMA Exercises. Enunciado equivalente del ejercicio de formulación; la numeración del repositorio en línea es diferente. https://aimacode.github.io/aima-exercises/search-exercises/ex_8/

AIMA Exercises. Enunciado equivalente del problema de obstáculos poligonales. https://aimacode.github.io/aima-exercises/search-exercises/ex_9/

[2] De Berg, M., Cheong, O., Van Kreveld, M. y Overmars, M. (2008). Computational Geometry: Algorithms and Applications, 3.ª edición, sección 15.1, páginas impresas 324–325. https://www.cs.cmu.edu/afs/cs/academic/class/15456-s14/Handouts/BKOS.pdf

[3] Hauser, K. Robotic Systems. Motion Planning in Simple Geometric Spaces, apartado Visibility graph. https://motion.cs.illinois.edu/RoboticSystems/GeometricMotionPlanning.html

Fuentes web consultadas el 3 de noviembre de 2026. Las formulaciones, los costos elegidos y las comprobaciones son desarrollo de esta solución.